

Model-Based Reinforcement Learning

Put the model back — learned — and let the two lineages reunite

Jinkyoo Park · KAIST

A model given (7, 9), a model deleted (8, 10), a model *learned* — and shared

— HANDOFF

The handoff — the model returns

Two lectures threw the model away, from opposite traditions. This one puts it back — learned, and therefore wrong.

Where we are — both lineages, each missing its model

STAGES	dynamic ○ — ●	MODEL	data-driven ○ — ●	AGENTS	single agent ● — ○
--------	---------------	-------	-------------------	--------	--------------------

	LINEAGE A • OR / DYNAMIC PROGRAMMING	LINEAGE B • CONTROL THEORY
MODEL-BASED (ORIGIN)	MDP & Dynamic Programming LECTURE 7	Optimal Control / Planning LECTURE 9
DATA-DRIVEN (EXTENSION)	Value-Based RL LECTURE 8	Policy-Based RL LECTURE 10

No cell of the grid is highlighted, because **all four are ours today**. No axis moves either: still *dynamic*, *data-driven*, *single agent*. What changes is that we stop pretending the transition is unknowable. We **estimate it**, and every model-based method of Lectures 7 and 9 becomes available again — standing, this time, on data.

Under all four cells, one new row: **learn the model, and let the four collaborate**.

What each of the four gave up, and what it cost

Lecture 8 deleted P and R and kept the Bellman equation. Lecture 10 deleted f and kept the feedback law. Both paid in the same coin:

Sample inefficiency. Each transition is used once and thrown away.

Lecture 10 closed by asking the question both extensions had been avoiding: *if deleting the model cost us this much, what happens if we learn it?*

Here is the cost, measured. On a MuJoCo half-cheetah, a model-free learner needs of order 10^7 – 10^8 environment steps to reach a competent gait

— **about ten days of real time**. A model-based learner reaches a walking gait from 10^4 – 10^5 steps: **about ten minutes**. (Nagabandi et al., 2018)

The reason is arithmetic, not cleverness. A learned f_θ can be queried without touching the world, so **one real transition can inform the value of many imagined states**. That is the entire case for this lecture.

The thesis — learn the dynamics, and both parents come back

Learn the dynamics from data, and **the model-based machinery of both lineages comes back to work on top of it.**

Four chapters have circled one problem. Dynamic programming (Ch 7) and optimal control (Ch 9) *had* a model and computed. Value-based (Ch 8) and policy-based RL (Ch 10) *lacked* one and sampled. Model-based RL learns f_θ and lets them collaborate — and, most strikingly, has an optimal-control **teacher** train a policy **student**.

The price is **model bias**. Errors in f_θ compound through planning, and a planner is an optimiser, so it will find them. Managing that trade — sample efficiency against model error — is the engineering of the whole lecture, and it is Lecture 5's warning arriving in the dynamic world.

The unknowns doubled — and this is the third answer

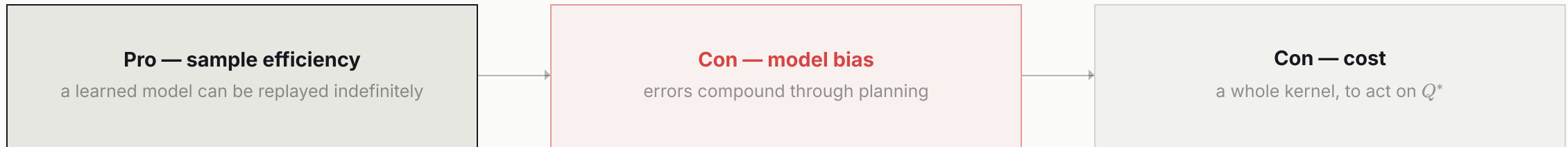
Lecture 0 named the deepest move in the course: crossing the model axis in the dynamic world leaves **two** unknowns, r and P , not one. Part IV is three answers to that doubling.

ANSWER	WHAT IT DOES WITH R AND P	CHAPTER
Value-based	fuses them into Q and models neither	Lecture 8
Policy-based	lets P vanish from the gradient (the score-function trick)	Lecture 10
Model-based	estimates P explicitly , then plans	Lecture 11

Read the third row against the first two. It is the only one that pays for a whole transition kernel — far more parameters than the Q^* we act on. Lecture 8 called that waste and walked past it. Today we collect the return on that investment: **a model you can replay**.

Lecture 8 promised this lecture, in three lines

Backup 4 of Lecture 8 set model-based RL aside with a ledger. Here it is, and here is where each line is paid:



- the **pro** is Acts 1 and 4 — the spectrum, and Dyna's imagined steps;
- the **con of bias** is Act 2 — replanning, ensembles, and uncertainty-aware models;
- the **con of cost** is answered by Act 3, which stops paying for accuracy the task does not use.

And Lecture 10 leaves something more specific: [the trust-region machinery](#). It returns in Act 4, unchanged, as the constraint $D_{\text{KL}}(p(\tau) \parallel \bar{p}(\tau)) \leq \epsilon$ that keeps a trajectory optimiser inside the region where its local model is believable. Same object, third appearance.

The roadmap — four questions



- **Q1 — Why bring a model back at all?** Sample efficiency, and [the spectrum](#) it opens between model-free and fully differentiable control.
- **Q2 — How do we use a learned model?** [Planning](#) over f_θ — Lecture 9's optimal control, run on learned dynamics, and re-planned every step.
- **Q3 — Can the planner itself be trained?** [Differentiable MPC](#) — backpropagate *through* the optimiser, and fit the model to the task.
- **Q4 — How do the lineages reunite?** Three routes from a model to a policy, of which [guided policy search](#) is the clearest: control teaches, a network learns. (*Levine & Koltun, 2013*)

— ACT 1

The spectrum of model use

Q1. "Model-based" is not the opposite of "model-free". It is the far end of a dial.

Two ways to come by a model



OFTEN WE SIMPLY HAVE ONE

- **Games** — Atari, chess, Go: the rules *are* the transition function.
- **Easily modelled systems** — a car following a bicycle model.
- **Simulated environments** — the simulator is the dynamics.

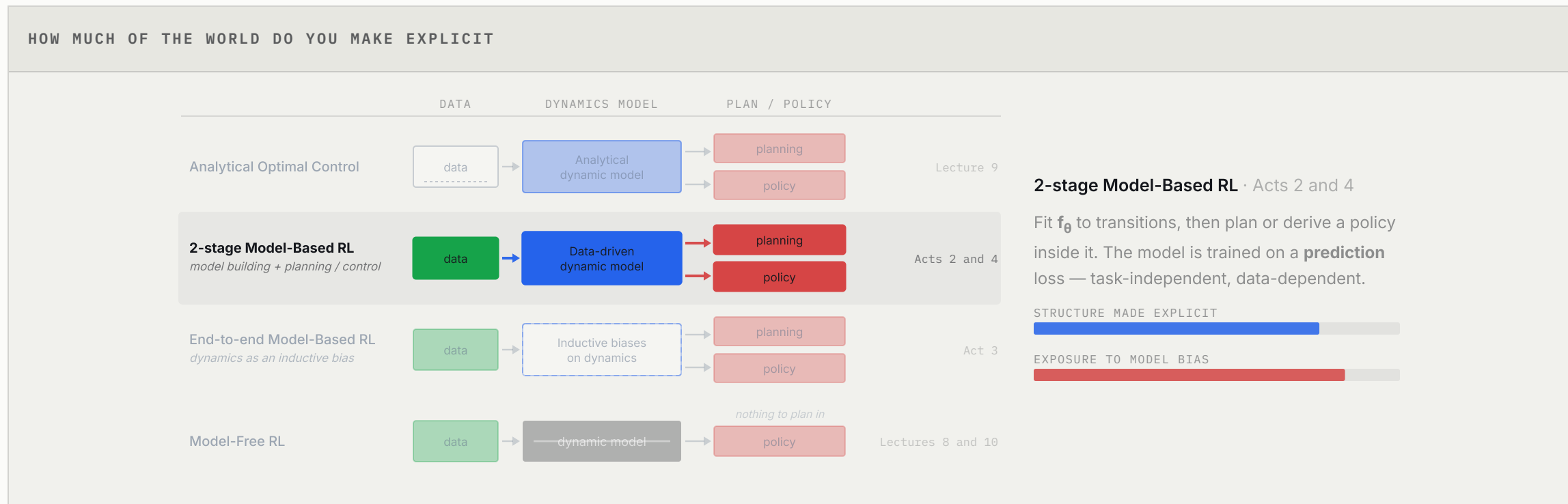
OFTEN WE CAN LEARN ONE

- **System identification** — fit the few unknown parameters of a **known** structure. Excite the system, measure the response, fit.
- **Learning** — fit a **general-purpose** model to observed transitions $\{(s, a, s')\}$, with no structure assumed.

Either way, the payoff has two faces, and they are Acts 2 and 4 of this lecture:

Knowing the dynamics helps you **plan a sequence of actions**, and it helps you **derive a policy**.

The ladder of model use



Four ways to get from data to a decision, drawn as the source deck draws them. Walk down and the model is made less and less explicit: an analytical model needing no data at all; a model *fitted* then planned in; dynamics baked into the architecture as an inductive bias; and finally the model deleted, data wired straight to a policy. Note the right-hand column: only the model-free row cannot produce a **plan**, because there is nothing to plan *in*.

The same ladder, as a ledger

	MODEL-FREE RL	MODEL-BASED RL	DIFFERENTIABLE MPC
explicit model?	no	yes	yes
trained object	policy $\pi(\cdot)$	model $f(\cdot)$	model $f(\cdot)$, and cost $c(\cdot)$
loss	task loss	model-prediction loss	task loss
execution	policy	planning	planning

Read left to right and more of the world's structure is made explicit, buying **sample efficiency** and generalisation. Read right to left and there is less to get wrong, dodging **model bias**.

A learned model is reusable but **wrong**; a model-free policy is honest but **hungry**.

The price, named — the model is only right where the data was

The naive plan is to fit a model, plan in it, and execute. It works for exceptionally simple cases — those in which the fitted model generalises over the *whole* state space.

WHAT GOES WRONG

A model is fitted where the base policy went. Planning then evaluates it where the *plan* goes, which is somewhere else — and an optimiser will walk straight to wherever the model is most flatteringly wrong.

$$p_{\pi_f}(s_t) \neq p_{\pi_0}(s_t)$$

WHERE YOU HAVE MET THIS BEFORE

Lecture 5, exactly: a surrogate fitted to a fixed dataset, then handed to an optimiser that exploits it. **The optimiser is an adversary** — and a planner is an optimiser with a time axis.

And it gets *worse* with capacity: distribution mismatch is **exacerbated as the model class becomes more expressive**.

So the optimum found in a learned model is only **optimum from the model's point of view**. The whole of Act 2 is a sequence of four repairs to that one sentence.

— ACT 2

Planning with a learned model

Q2. Fit a model; plan in it. Four versions, each named by the failure of the last.

Version 1 — fit, then plan once

Q1 Why a model?

Q2 Plan with it?

Q3 Differentiable?

Q4 Reunite?

MODEL BUILDING + OPEN-LOOP PLANNING

1. Run a base policy $\pi_0(a_t | s_t)$ — e.g. random — and collect $\mathcal{D} = \{(s, a, s')_i\}$ (*experiencing*)
2. Learn $f(s, a)$ by minimising $\sum_i \|f(s_i, a_i) - s'_i\|^2$ (*model fitting*)
3. Plan through $f(s, a)$ to choose actions (*model-based planning*)
4. Execute those actions

YES — THIS IS CLASSICAL SYSTEM IDENTIFICATION

Excite, measure, fit a **predefined** model. Particularly effective when physics gives the structure and only a few parameters are free: fit Newton's law once and it describes any rigid body.

Linear sysID estimates A, B in $\dot{x} = Ax + Bu$ — and **once the linear system is identified, all of Lecture 9 applies**: Riccati, K , the lot.

NO — THE DISTRIBUTION MOVES UNDER YOU

The plan is executed open loop. The states it visits are not the states the model was fitted on, and there is no correction channel.

$$p_{\pi_f}(s_t) \neq p_{\pi_0}(s_t)$$

What system identification actually offers

Before reaching for a network, note how much structure the control tradition already knows how to fit — and that every one of these was built to be *planned in*.

METHOD	MODEL	WHAT IT BUYS
Linear sysID	$\dot{x} = Ax + Bu$	Lecture 9 applies verbatim
Model reduction / DMD	$\dot{a} = \tilde{A}a + \tilde{B}u$	a low-dimensional state — fast prediction, fast control
Piecewise linear	local A_k, B_k	iLQR's local model, made explicit
Koopman operator	linear in infinite-dimensional measurements	a nonlinear system, solved by linear theory
Deep / GP models	MLP, RNN, LSTM, Gaussian process	expressive — and the reason distribution mismatch bites

The table is a spectrum of its own: downward, more expressive and less planable. Act 3 will argue the direction of travel should sometimes be *upward*.

Version 2 — put the loop back

Append what actually happened, refit, repeat. If the model is wrong where the plan goes, then [go there and collect data](#):

ITERATIVE MODEL BUILDING

5. Append $\{(s, a, s')_j\}$ to $\mathcal{D}$, and return to step 2 (*collecting data*)

This is data aggregation: chase p_{π_f} with the dataset until $p_{\pi_f}(s_t) = p_{\pi_0}(s_t)$. It helps — and it still executes a whole plan before looking.

So: what if we make a mistake [in the middle of the plan](#)?

Version 3 — re-plan every step, and MPC is born

At each step, optimise a short horizon of actions, **execute only the first**, observe, re-plan:

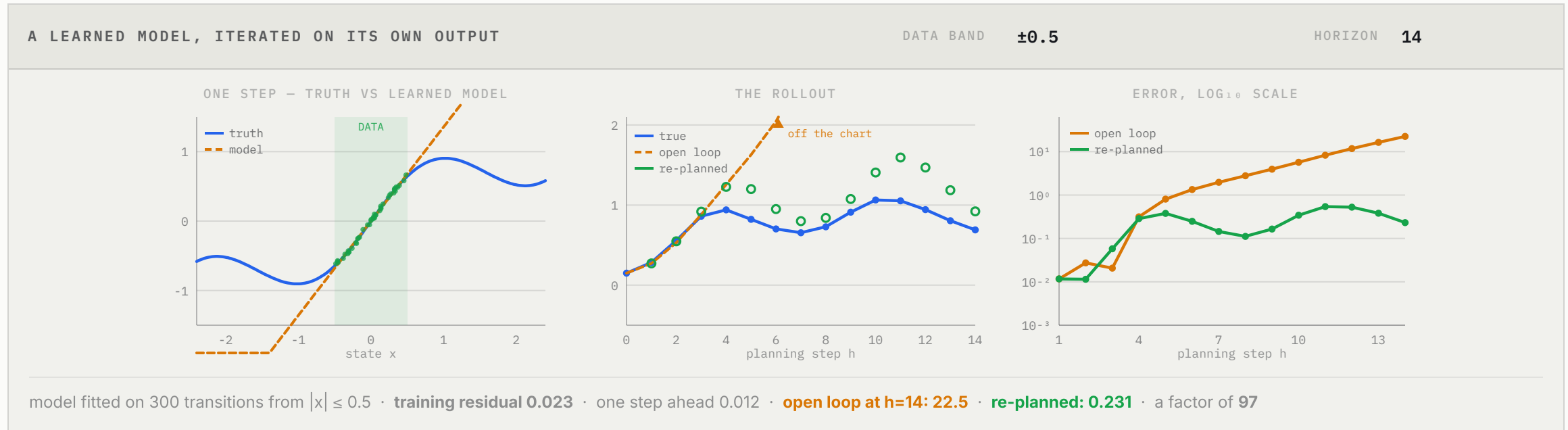
$$\tau_{1:T}^* = \text{MPC}(x_{\text{init}}; C, f_\theta) = \arg \min_{x_{1:T}, u_{1:T}} \sum_{t=1}^T C_t(x_t, u_t) \quad \text{s.t.} \quad x_{t+1} = f_\theta(x_t, u_t), \quad x_1 = x_{\text{init}}$$

This is **Lecture 9's optimal control, run on a learned model** instead of a given one. Nothing in the solver changes; only the constraint is now fitted.

REPLANNING HELPS WITH MODEL ERRORS

- the more you re-plan, **the less perfect each individual plan needs to be**;
- you can therefore use **shorter horizons**, where the model is still trustworthy;
- and the inner optimiser can be crude — even random shooting often works.

What a rollout does to a model's error



A one-dimensional system, a base policy that explored a band around the origin, and a model fitted inside it. Left: the model is **excellent where the data is** and hopeless outside. Middle: rolled out open loop, the model is composed with *its own output*, so it walks out of the band and never comes back. Right, on a log axis: the open-loop error climbs a decade every few steps, while **re-measuring the state each step holds it flat**.

Why it compounds, precisely

Iterating a one-step model means feeding it inputs it never saw — its own predictions.

$$\hat{y}_{t+k} = f(\hat{x}_{t+k}) + e_{t+k}, \quad \hat{x}_{t+k} = (\underbrace{\hat{y}_{t+k-1}, \dots, \hat{y}_{t+1}}_{\text{random variables}}, \underbrace{y_t, \dots, y_{t+k-L}}_{\text{data}})$$

As the horizon grows the model's input is made of **more of its own guesses and less of the world**. Each step's error is the next step's input error, and the two multiply.

Two consequences the practice lives by. Keep the planning horizon short. And re-plan often, because re-planning replaces the leading random variables with data — which is exactly what the widget's flat green line is.

Version 3+ — plan with the model's uncertainty

Version 3 still acts on the model's *mean*: it takes actions believed good in expectation, which quietly forbids exploring the places the model does not know.

Split the error the way Lecture 0 split it, and the way Lecture 2 wrote it:

$$\mathbb{E}\|Y - \hat{f}(X)\|^2 = \underbrace{\mathbb{E}\|Y - f(X)\|^2}_{\sigma_s^2 = \text{systemic noise}} + \underbrace{\mathbb{E}\|f(X) - \hat{f}(X)\|^2}_{\sigma_m^2 = \text{model uncertainty}}$$

The first term is irreducible. The second is **high exactly where training data is thin** — and it is the quantity a planner must respect.

Three ways to get σ_m , all of them borrowed

ENSEMBLES

$$p(\theta \mid \mathcal{D}) \approx \frac{1}{N} \sum_i \delta(\theta_i), \quad \int p(s' \mid s, a, \theta) p(\theta \mid \mathcal{D}) d\theta \approx \frac{1}{N} \sum_i p(s' \mid s, a, \theta_i)$$

Train each θ_i on $\mathcal{D}_i$, sampled with replacement. [Lecture 5's ensemble-alarm](#), in dynamics.

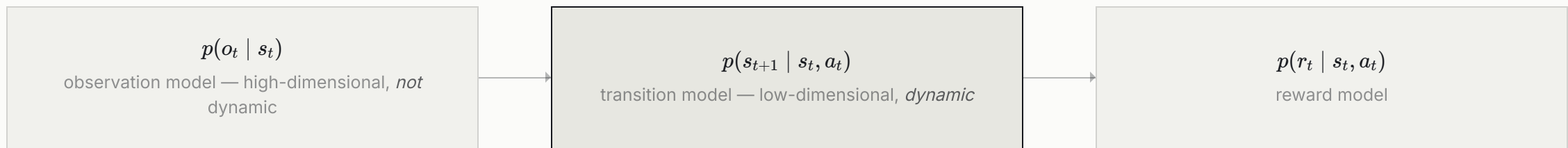
GAUSSIAN PROCESSES

A GP transition model is *non-parametric* (nonlinear dynamics), *Bayesian* (non-stationary dynamics — update on recent data), and *probabilistic* (robust decisions). [Lecture 4's posterior](#), used as f .

The third is a Bayesian network — put a distribution on the weights rather than a point estimate (*Blundell et al., 2015*). All three answer the same question Lecture 2 asked about a coin: **do not carry a number, carry a belief**. **PETS** combines the first two — a probabilistic *ensemble*, whose spread far from data is epistemic and whose per-member variance is aleatoric — and learns four continuous-control tasks in [under 100 000 steps, or 100 trials](#), where PPO, SAC and DDPG need one to two orders of magnitude more. (*Chua et al., 2018*)

Version 4 — plan in a latent space

Raw observations are high-dimensional, redundant and partial. The dynamics are not.



Learn all three at once, with a deterministic encoder $q_\psi(s_t | o_t) = \delta(s_t = g_\psi(o_t))$:

$$\max_{\phi, \psi} \frac{1}{N} \sum_i \sum_t \underbrace{\log p_\phi(g_\psi(o_{t+1,i}) | g_\psi(o_{t,i}), a_{t,i})}_{\text{latent dynamics}} + \underbrace{\log p_\phi(o_{t,i} | g_\psi(o_{t,i}))}_{\text{reconstruction}} + \underbrace{\log p_\phi(r_{t,i} | g_\psi(o_{t,i}))}_{\text{reward}}$$

This is [Lecture 6's VAE, carrying a transition](#). And the payload of *Embed to Control* is worth naming: it learns a latent space in which the dynamics are locally [linear](#), so Lecture 9's LQR applies to a robot controlled from pixels. (*Watter et al., 2015; Zhang et al., 2019*)

A model built to be planned in — convexity, learned

Every version so far fits a model and hopes the planner copes. Turn the question round: what property should a learned model *have* so that planning inside it is easy?

Lecture 1's answer, imposed on a network. An **input-convex** neural network is convex in its input by construction — every $W^{(z)}$ non-negative, every activation convex and non-decreasing — so the predictive-control problem

$$\min_{u_t, \dots, u_{t+T}} \sum_{\tau} f(x_{\tau-n_w}, \dots, x_{\tau}) \quad \text{s.t.} \quad s_{\tau} = g(x_{\tau-n_w}, \dots, u_{\tau}), \quad \underline{u} \leq u_{\tau} \leq \bar{u}, \quad \underline{s} \leq s_{\tau} \leq \bar{s}$$

is **a convex program in the actions**, solvable to global optimality. (*Amos, Xu & Kolter, 2017; Chen, Shi & Zhang, 2019*)

THE BILL, PAID IN A REAL BUILDING

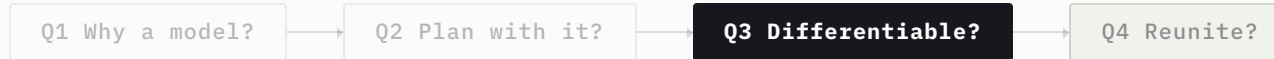
An input-convex recurrent model fits building dynamics as accurately as an ordinary RNN, and the controller built on it finds actions worth **11.52% more energy saving** — while the ordinary RNN's decisions "vary dramatically". Later work ran the same controller on a real ETH building for a fortnight. **Lecture 1's convexity was never a mathematical convenience; it is what makes a learned model safe to optimise inside.**

— ACT 3

Differentiable control

Q3. The model is fitted to predict. But it is *used* to decide. Fit it for what it is used for.

Treat the planner as a policy class



Every version in Act 2 fits f_θ by minimising $L(x_{t+1}, f_\theta(x_t, u_t))$ — a loss that is **task-independent and data-dependent**. Accuracy is therefore spent uniformly, and since the model class is wrong *somewhere*, prediction loss chooses where on the data's terms rather than the task's.

But $\text{MPC}(\cdot)$ maps a state to an action, which is what a policy is. So fit its parameters end to end on a task loss — for instance, imitating an expert controller $\hat{\theta}$:

$$\mathcal{L}(\theta) = \mathbb{E}_x \left[\left\| u_{1:T}(x; \theta) - u_{1:T}(x; \hat{\theta}) \right\|^2 \right]$$

The trained object is still the model, the execution is still planning — only the loss moves from **prediction** to **task**.

Is the $\arg \min$ differentiable? Partially, yes

For a convex quadratic planning subproblem — an LQR, or a QP —

$$\tau_{1:T}^* = \arg \min_{\tau_{1:T}} \sum_t \frac{1}{2} \tau_t^\top C_t \tau_t + c_t^\top \tau_t \quad \text{s.t.} \quad x_1 = x_{\text{init}}, \quad x_{t+1} = F_t \tau_t + f_t$$

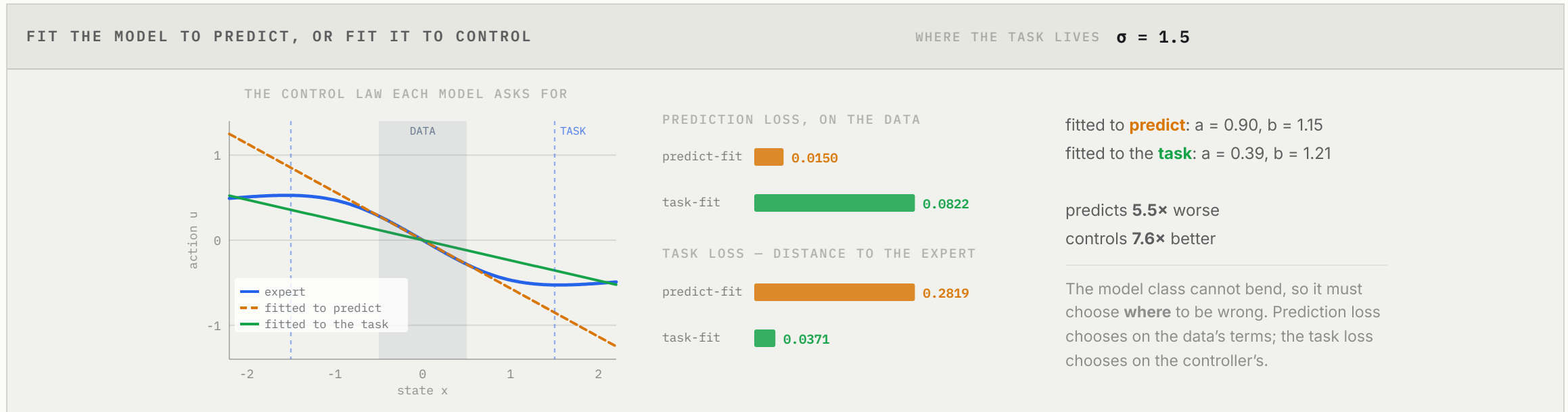
the optimum is characterised by its KKT conditions. Do not differentiate τ^* ; **differentiate the conditions it satisfies**:

$$\mathcal{L}(\tau, \lambda) \implies \text{KKT} \implies \text{take differentials} \implies \frac{\partial \tau^*}{\partial \theta}$$

The differentials give a *linear system* in $(d\tau, d\lambda)$ whose matrix is the KKT matrix already formed in the forward pass — so the backward pass is one more solve of the same structure. For LQR that solve **is itself an LQR problem**: one Riccati sweep. (*Amos & Kolter, 2017; Amos et al., 2018*)

And the saving is measurable: differentiating the fixed point costs **about 1.5×10^{-2} s regardless of horizon**, while unrolling the iLQR solver and backpropagating through every iteration costs ≈ 3 s at 128 steps — two orders of magnitude, and growing.

A model fitted to be wrong in the right places



The same misspecified model class, fitted twice: once to minimise prediction error on the data, once to minimise the *task* loss by differentiating through the one-step planner. Move the task away from where the data was collected and the two fits separate — the task-fitted model becomes a **worse predictor and a better controller**. Both losses are printed; neither is asserted.

Why it matters, and where the evidence is

- **Imitation learning.** Fit θ so that $\pi_\theta(s) \approx \hat{\pi}(s)$, inheriting a controller's structure while learning from demonstrations rather than from a hand-written cost.
- **Structure as inductive bias.** The constraint $x_{t+1} = f_\theta(x_t, u_t)$ is **baked into the architecture**, so far less data is needed than for a generic policy network, and extrapolation improves.
- **Task-aligned models.** Accuracy lands where the task needs it, which is the direct repair of Act 2's "prediction-accurate, control-useless" failure.

MEASURED, ON A DELIBERATELY MISSPECIFIED PENDULUM

The expert's dynamics lie outside the learner's model class. Against a plain system-identification baseline, the task-loss method ends with a *higher system-ID loss (0.0025 vs 0.0020)* and a *lower imitation loss (0.12 vs 0.19)*. A model fitted worse, in the way the task does not care about, controls better.

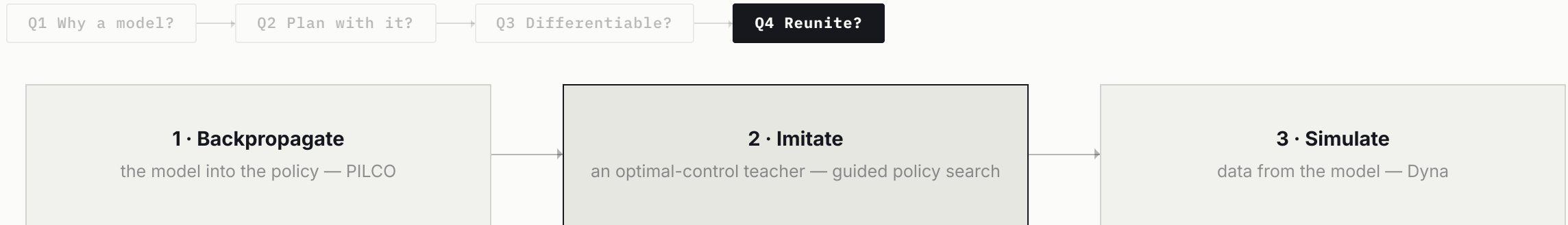
The honest caveat: for nonlinear problems this needs iLQR to reach a fixed point, and sometimes it does not — then you are back to unrolling the solver. The controller becomes **a differentiable layer**, and like every layer it has conditions of use.

— ACT 4

The lineages teach each other

Q4. A learned model gives you a plan. How do you get a *policy* — and whose policy is it?

Three routes from a model to a policy



Each route is one of the course's earlier chapters, [feeding its own data-driven descendant](#).

ROUTE	WHO TEACHES	WHO LEARNS
backpropagate	Lecture 9's shooting view — a chain of Jacobians	the policy parameters θ
imitate	Lecture 9's iLQR, as an oracle	Lecture 10's π_θ
simulate	Lecture 7's planning backup	Lecture 8's Q -learning

Route 1 — backpropagate the model into the policy

Compose policy and model along the horizon and differentiate the whole chain:

$$a_t = \pi_\theta(s_t) \rightarrow s_{t+1} = f(s_t, a_t) \rightarrow a_{t+1} = \pi_\theta(s_{t+1}) \rightarrow \dots, \quad \nabla_\theta J = \sum_i \frac{dr_t}{ds_t} \prod_{t'=2}^t \frac{ds_{t'}}{da_{t'-1}} \frac{da_{t'-1}}{ds_{t'-1}}$$

PILCO does this with a *probabilistic* model — a GP on the state difference $\Delta_t = x_t - x_{t-1}$ — and propagates the whole distribution forward by moment matching, so long-horizon planning carries the model's own uncertainty. Policy evaluation is then closed form and the policy gradient is analytic. (*Deisenroth & Rasmussen, 2011*)

The result is the data-efficiency headline of the field: real cart-pole swing-up *and* balance from **17.5 seconds of interaction with the physical hardware**; a robotic unicycle in $\mathbb{R}^{12}$ from about 20 trials. Against the methods of the decade before it, roughly **three orders of magnitude less interaction**.

Why route 1 is not the answer

WHAT PILCO FIXED

Model bias is worst when data is scarce and no prior structure is available: many different deterministic functions fit the same handful of transitions, and they disagree wildly between the points. A **probabilistic** model reports that disagreement instead of picking one curve, and planning can then respect it.

WHAT REMAINS BROKEN

- **Parameter sensitivity** — the same pathology as shooting methods. Policy parameters **couple every time step**, so there is no LQR-like second-order structure and no dynamic programming to exploit.
- **Vanishing and exploding gradients** — a product of many Jacobians, exactly as in backpropagation through time. And unlike an LSTM, **we cannot choose a convenient dynamics; nature chose it.**

Note the contrast with Lecture 10. The policy gradient's score-function form has **no product of Jacobians at all** — the dynamics vanished. With enough samples it is the more stable estimator. That is why route 3 exists.

Route 2 — let optimal control teach

Lecture 1 wrote an optimisation. Lecture 9 added the dynamics as a constraint. Add one more constraint and you have this lecture.

	OBJECTIVE	DYNAMICS CONSTRAINT	POLICY CONSTRAINT
Optimisation (<i>Lec 1</i>)	$\min_u c(u)$	—	—
Optimal control (<i>Lec 9</i>)	$\min_{u,x} \sum_t c(x_t, u_t)$	$x_t = f(x_t, u_t)$	—
+ imitation (<i>Lec 11</i>)	$\min_{u,x,\theta} \sum_t c(x_t, u_t)$	$x_t = f(x_t, u_t)$	$u_t = \pi_\theta(x_t)$

Three rows, three lectures. The middle row constrains the trajectory to be *physical*; the bottom row constrains it to be **reproducible by a policy**. Everything in guided policy search follows from that one extra line.

Guided policy search — solve it by alternation

The constrained program, with an augmented Lagrangian:

$$\bar{\mathcal{L}}(\tau, \theta, \lambda) = c(\tau) + \sum_t \lambda_t (\pi_\theta(x_t) - u_t) + \sum_t \rho_t (\pi_\theta(x_t) - u_t)^2$$

1 • Trajectory optimisation

$$\tau^* \leftarrow \min_\tau \bar{\mathcal{L}} \text{ — via iLQR}$$

2 • Supervised fit

$$\theta^* \leftarrow \min_\theta \bar{\mathcal{L}} \text{ — via SGD}$$

3 • Dual update

$$\lambda \leftarrow \lambda + \alpha d\bar{\mathcal{L}}/d\lambda$$

Step 1 is Lecture 9. Step 2 is plain supervised learning. Step 3 is dual gradient descent — and its derivation uses the same envelope argument as Act 3: at the inner optimum $d\mathcal{L}/dx^* = 0$, so **the gradient through the arg min collapses to a single term**.

And the direction of teaching is not one-way: **the optimal-control teacher adapts to the learner**, avoiding actions the student cannot mimic.

What makes it work — and where the trust region reappears

In the stochastic form, the local controller is constrained to stay near the previous one:

$$\min_p \sum_t \mathbb{E}_{p(x_t, u_t)} [c(x_t, u_t)] \quad \text{s.t.} \quad D_{\text{KL}}(p(\tau) \parallel \bar{p}(\tau)) \leq \epsilon, \quad p(u_t \mid x_t) = \pi_\theta(u_t \mid x_t)$$

$$p(u_t \mid x_t) = \mathcal{N}(K_t(x_t - \hat{x}_t) + k_t + \hat{u}_t, \Sigma_t)$$

Look at the second line. The local policy *is* [Lecture 9's](#) $u = Kx$, with a Gaussian around it — which is why optimising the trajectory hands you the local controller for free, out of the LQR structure.

And the first line is [Lecture 1's trust region, third appearance](#): a constraint that keeps each step inside the region where the local model is believable. Lecture 1 measured that region in $\|x - x^{(k)}\|$, Lecture 10 in KL between policies, and here in KL between *trajectory distributions*. Same ratio-and-restrict logic every time.

Because time-varying linear-Gaussian dynamics are a strong and reasonable *local* assumption for a physical system, each trajectory is optimised from very few samples: [5 to 20](#) where REPS, CEM and RWR need 100 to 800, and nine real manipulation tasks — stacking lego, threading a ring, screwing a bottle cap — converge in about [40 samples each](#). Policy search has become supervised learning against an oracle. (*Levine & Abbeel, 2014; Levine et al., 2016*)

The teacher that watches the student — PLATO

The naive version of route 2 collects a dataset from an MPC expert and fits the policy to it. That ignores the one thing imitation always breaks on.

The states visited by the *teacher* are not the states visited by the *learner*, so nothing guarantees long-horizon performance. **PLATO** repairs it by pulling the teacher toward the student at every step:

$$\pi_{\lambda}^t(u \mid x_t, \theta) \leftarrow \min_{\pi} J_t(\pi \mid x_t) + \lambda D_{\text{KL}}(\pi(u \mid x_t) \parallel \pi_{\theta}(u \mid o_t))$$

The only difference from ordinary MPC is that KL term. It makes the teacher visit the states the student will visit, while still reacting competently to surprises the half-trained student could not survive.

Two practical consequences. The MPC teacher may use **full state** at training time while the final policy uses **only the observations** the robot will have at test time — the input-remapping trick. And in flight experiments the crash count stays near zero throughout training, where DAGger's saturates: this is a **safety** argument, not only an accuracy one. (*Kahn et al., 2017*)

Route 3 — imagine the data

The simplest reunion of all: use the model to *manufacture experience*, and hand it to any model-free algorithm unchanged.

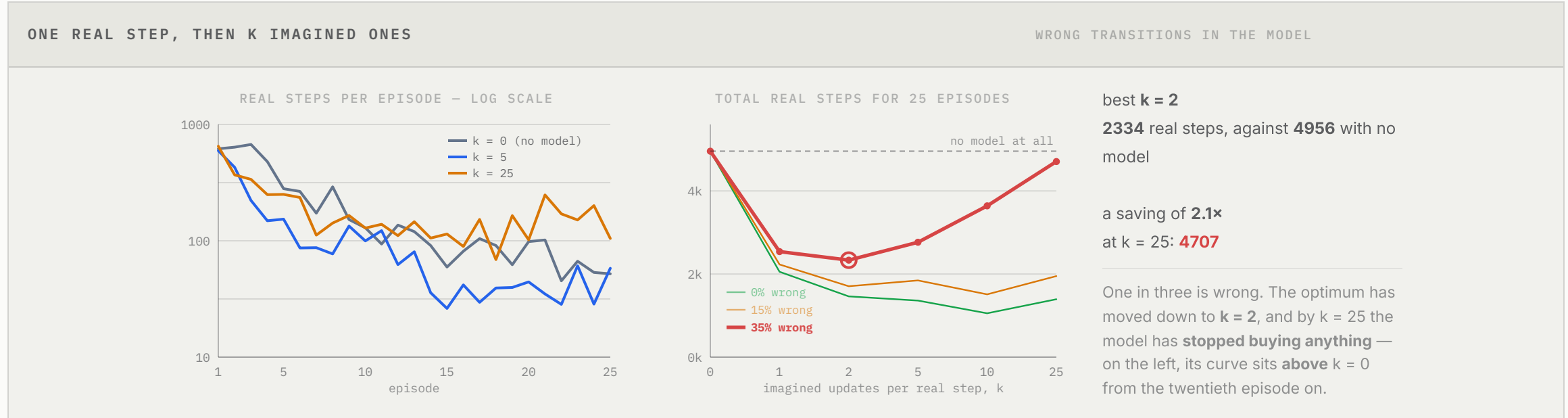
DYNA — ONLINE Q-LEARNING THAT PERFORMS MODEL-FREE RL WITH A MODEL

1. From s , pick a by an exploration policy; observe (s, a, s', r)
2. Update the model $\hat{p}(s' \mid s, a)$ and $\hat{r}(s, a)$
3. $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ (from real experience)
4. Repeat k times: sample (s, a) from the buffer, **simulate** $s' \sim \hat{p}, r = \hat{r}$, and apply the same update (from imagined experience)

Line 3 is Lecture 8, untouched. Line 4 is Lecture 7's planning backup, on an estimated model. **The two lineages meet inside a single loop, four lines apart.** (Sutton, 1990)

Only short rollouts are needed — as few as one step — and the algorithm still sees diverse states, because the imagined transitions start from every state in the buffer. Longer rollouts from π give MVE and MBPO; the question their titles ask is the honest one: **when to trust your model.**

Real steps against imagined steps



Sutton's maze, learned three ways. Raise k — the number of imagined updates per real step — and the number of real environment steps needed collapses. Then corrupt a fraction of the learned model's transitions and raise k again: past a point the imagined updates **stop buying anything at all**, and the heavily-planning agent ends up behind the one with no model. The gain and its price, on one screen.

The hazard, admitted by the method's own authors

WHAT BREAKS

Imagination rollouts "can suffer from **severe bias when the learned model is inaccurate**". In continuous control it proved "very difficult to train nonlinear neural network models for the dynamics that would actually improve the efficiency of Q-learning when used for imagination rollouts."

The fix that worked was to give up expressiveness: **iteratively refitted time-varying linear dynamics**. (*Gu et al., 2016*)

WHAT IS WORTH KEEPING

The same paper answers Lecture 8's parting wall from the other side. Write Q as a value plus a quadratic advantage,

$$Q(x, u) = V(x) - \frac{1}{2}(u - \mu(x))^{\top} P(x)(u - \mu(x))$$

and the maximising action is **always $\mu(x)$** , analytically. The continuous $\arg \max$ is solved by **assuming LQR structure**: Lecture 9 answering Lecture 8 directly.

The reunion, and the limits

Three routes, one mechanism: a model-based method supplies the answers, a data-driven method learns to reproduce them, and the model underneath is *learned*.

THE WHOLE OF PART IV, IN ONE LINE

DP / optimal control (*model-based*) $\xrightarrow{\text{teaches}}$ value / policy (*data-driven*), on top of a **learned dynamics model**. Four chapters become one system.

And the honest ledger, which the source deck insists on:

- **You need a model** — not always available, and **sometimes harder to learn than the policy itself**;
- **Learning it costs time and data** — expressive classes are slow, fast classes are inexpressive;
- **It relies on assumptions** — linearisability, continuity, smoothness.

Route 1 is simple but unstable; route 2 is sample-efficient but needs a real planner (iLQR, MCTS, MPC); route 3 is simple but the least sample-efficient of the three.

And design optimisation returns — bilevel

Lecture 1 optimised a wind-farm layout with a trust region, and Lecture 5 designed from a fixed dataset. Neither could touch the *controller*. Differentiable control closes that loop.

Many real problems are **bilevel** — an upper-level *design* p wrapping a lower-level *control* problem:

$$\min_p \sum_t \mathcal{L}(x_{t+1}, u_t^*; p) \quad \text{s.t.} \quad u_{0:T-1}^* = \arg \min_u \sum_t \mathcal{L}(x_{t+1}, u_t; p), \quad x_{t+1} = f(x_t, u_t; p)$$

Turbine layout with the yaw controller that will run on it; furnace geometry with the recipe that will heat it. Act 3 supplies the missing derivative: **differentiate the optimal control through to the design, and optimise both together** rather than designing first and controlling afterwards.

The evidence is already in the room. An input-convex model inside an MPC loop saved **11.52%** of a building's energy and then ran on a real one; the differentiable-MPC literature names **HVAC control and furnace control** as its motivating applications. Lecture 1's **trust-region** widget solved the wind-farm layout by a staircase of convex problems with the controller held fixed. **Here the controller stops being fixed.**

— CLOSING

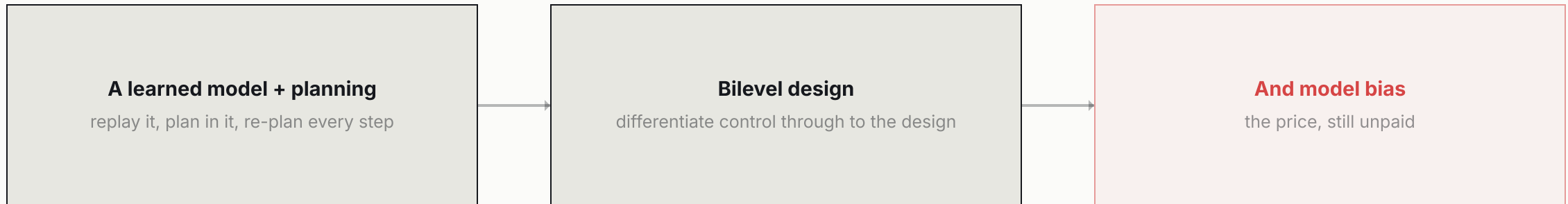
Closing

Where we are — Part IV complete

OR / DYNAMIC PROGRAMMING		CONTROL THEORY
Model-based	MDP & DP (<i>Lec 7 ✓</i>)	Optimal Control (<i>Lec 9 ✓</i>)
Data-driven	Value-Based RL (<i>Lec 8 ✓</i>)	Policy-Based RL (<i>Lec 10 ✓</i>)
Model-Based RL (<i>Lec 11 ✓</i>) — the four, reunited on a learned model		

We can decide statically (Ch 1–6) and dynamically (Ch 7–11); with a model and without; by searching values and by producing policies; and now by [learning the model](#) so that all of it can collaborate.

What we hand on



Every repair in this lecture depended on one privilege we never questioned: when the model was wrong, **we could go and collect the transition that proved it**. Version 2 aggregated data. Version 3 re-planned from a fresh measurement. Dyna interleaved imagined updates with real ones that corrected them.

Take that away — no new samples, ever — and model bias stops being a nuisance and becomes **the whole problem**.

That is Lecture 12, offline RL. Its answer will be this lecture's learned model, made **pessimistic**: penalise the reward by the model's own uncertainty, $\tilde{r}(s, a) = r(s, a) - \lambda u(s, a)$, and plan in that penalised MDP. (MOPO, Yu et al., 2020; MOREL, Kidambi et al., 2020) The dial you turned in Act 2 becomes the only defence left.

Model-free RL learned to act by forgetting the
model. Model-based RL *learns it back*.

To plan with it, to control through it, and to let dynamic programming and optimal control — the course's
two parents — finally teach each other.

Questions?

The arc of Part IV in three moves: a model **given** (7, 9), a model **deleted** (8, 10), a model **learned and shared** (11). What remains is to give up the last luxury — the right to try something and see what happens.

— APPENDIX

Backup slides

Complete arguments, kept out of the narrative.

Backup 1 — differentiating the QP / LQR arg min, in full

For a convex QP planning subproblem $\min_z \frac{1}{2} z^\top Q z + q^\top z$ subject to $Az = b$, the optimum is characterised by its KKT system:

$$Qz^* + q + A^\top \nu^* = 0, \quad Az^* - b = 0$$

Rather than differentiate z^* directly, take **differentials** of both conditions:

$$dQ z^* + Q dz + dq + dA^\top \nu^* + A^\top d\nu = 0, \quad dA z^* + A dz - db = 0$$

$$\Rightarrow \begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} dz \\ d\nu \end{bmatrix} = - \begin{bmatrix} dQ z^* + dq + dA^\top \nu^* \\ dA z^* - db \end{bmatrix}$$

One solve of this system yields $\partial z^* / \partial(\cdot)$ for every parameter, so the planner is a differentiable layer. Writing $[d_z; d_\nu]$ for its solution against $[(\partial \ell / \partial z^*)^\top; 0]$, the chain rule gives $\nabla_Q \ell = \frac{1}{2}(d_z z^\top + z d_z^\top)$, $\nabla_q \ell = d_z$, $\nabla_A \ell = d_\nu z^\top + \nu d_z^\top$ and $\nabla_b \ell = -d_\nu$.

The LQR specialisation. With $\tau = (x, u)$ the finite-horizon LQR optimum solves exactly such a system, whose matrix is block-tridiagonal in (τ_t, λ_t) — and because that matrix has LQR structure, the backward solve *is another LQR problem*, $d_{\tau_{1:T}}^* = \text{LQR}_T(0; C, \nabla_{\tau^*} \ell, F, 0)$: one Riccati sweep, reusing the forward pass's factorisations.

Nonlinear case: run iLQR to a fixed point, Taylor-expand there, and differentiate the resulting LQR, zeroing the rows of F for tight control constraints. If iLQR has no fixed point, fall back to unrolling. (*OptNet; Differentiable MPC*)

Backup 2 — guided policy search, the constrained program

Objective. Minimise trajectory cost while forcing the trajectory distribution to agree with the policy:

$$\min_{\theta, p(\tau)} \mathbb{E}_{p(\tau)} \left[\sum_t c(x_t, u_t) \right] \quad \text{s.t.} \quad D_{\text{KL}}(p(x_t)\pi_\theta(u_t | x_t) \| p(x_t, u_t)) = 0 \quad \forall t$$

Lagrangian and alternation. Form $\mathcal{L}_{\text{GPS}}(\theta, p, \lambda) = \mathbb{E}_{p(\tau)}[\ell(\tau)] + \sum_t \lambda_t D_{\text{KL}}(p(x_t)\pi_\theta(u_t | x_t) \| p(x_t, u_t))$ and minimise by turns:

1. **w.r.t.** $p(\tau)$ — trajectory optimisation by iLQG under time-varying linear-Gaussian dynamics; the LQR structure makes the local controller $p(u_t | x_t) = \mathcal{N}(K_t(x_t - \hat{x}_t) + k_t + \hat{u}_t, \Sigma_t)$ fall out automatically;
2. **w.r.t.** θ — supervised regression: minimise the weighted sum of KL divergences between π_θ and the local controllers;
3. **duals** λ — dual gradient descent (in practice often *scheduled* rather than updated).

Why the dual step is cheap. With $x^*(\lambda) = \arg \min_x \mathcal{L}(x, \lambda)$ and $g(\lambda) = \mathcal{L}(x^*(\lambda), \lambda)$, the chain rule gives $dg/d\lambda = (d\mathcal{L}/dx^*)(dx^*/d\lambda) + d\mathcal{L}/d\lambda$, and the first term vanishes because $d\mathcal{L}/dx^* = 0$ at the argmin. So $dg/d\lambda = d\mathcal{L}/d\lambda$ evaluated at x^* — no derivative through the inner solve is needed. That is the same envelope argument Act 3 exploits, used there in the case where the derivative through the solve *is* wanted. **Why it is sample-efficient.** Time-varying linear-Gaussian dynamics are a strong but reasonable *local* assumption for a physical system, so each trajectory is optimised with far fewer samples than general model-free search — the return on injecting control-theoretic structure.

Backup 3 — the four versions of model-based planning, side by side

	DATA	PLANNING	FIXES	STILL BROKEN
V1 Model building + open-loop planning	one batch from π_0	plan once, execute all	nothing	$p_{\pi_f} \neq p_{\pi_0}$
V2 Iterative model building	aggregate π_f 's own visits	plan once, execute all	the <i>training</i> distribution	a mistake mid-plan is never corrected
V3 + MPC	aggregate; refit every N steps	plan, execute the first action, re-plan	mid-plan errors; short horizons suffice	acts on the mean — never explores
V3+ + model uncertainty	as V3	plan against an ensemble / GP posterior	over-confidence off-distribution	cost, and the horizon still compounds
V4 + latent space	observations (o, a, o')	plan in the learned latent state	high-dimensional, partial observations	everything above, plus representation error

Propagating uncertainty through a horizon. If the state input is itself uncertain, $x \sim \mathcal{N}(\mu_x, \Sigma_x)$, then $p(f(x) \mid \mu_x, \Sigma_x) = \int p(f(x) \mid x, \mathcal{D}) p(x) dx$, which is intractable and is handled either by Monte-Carlo — sample $x^t \sim p(x)$ and average $p(f(x^t) \mid x^t, \mathcal{D})$ over T particles — or by exact moment matching to a Gaussian, which is PILCO's choice. Rolling a large number of such trajectories and taking their mean and standard deviation gives the fan of predictions that widens with horizon: **compounding uncertainty, drawn.**

Backup 4 — the sample-efficiency versus model-bias trade, made precise

The gain. A learned model can be queried without environment interaction, so one real transition informs the value or policy at *many* imagined states. Empirically this cuts the required real samples by orders of magnitude — decisive whenever interaction is expensive, slow or dangerous: robots, chemical processes, buildings, patients.

The hazard. Planning trusts f_θ . Where f_θ is wrong the planner optimises a fiction, and errors **compound** over the horizon because each step's error is the next step's input. This is the dynamic cousin of Lecture 5's overestimation trap: an optimiser exploiting a model's blind spots.

Backup 4b — the five mitigations, and where each appeared

MITIGATION	MECHANISM	WHERE
short planning horizons	fewer compositions of the model with itself	Act 2, V3
re-plan every step	replace predicted inputs with measured ones	Act 2, V3
ensembles and GP posteriors	plan pessimistically where the model disagrees with itself	Act 2, V3+
task loss instead of prediction loss	be accurate where the task needs it	Act 3
structural inductive bias	convexity, linear latents, local linear-Gaussian models	Acts 2 and 4

The through-line of the whole course: [respect the model's uncertainty, or it will be weaponised against you](#). Lecture 5 said it about a surrogate over designs; this lecture says it about a surrogate over dynamics; Lecture 12 will say it once more, about a value function, with the escape hatch of fresh data closed.